VIEWER PRODUCT

Technical Architecture

The document viewer as a product a third-party common data environment embeds.

Status. The rendering core is built and boundary-enforced, and so is the integration surface: the `cde.viewer.v1` embed protocol is implemented on both sides and tested against an independent host, the backend carries the `frame-ancestors` allow-list that used to refuse every frame, and the image serves the embed document itself. The demo host drives the whole protocol against it end to end. What has not happened is a real CDE integrating it.

Since the last issue, the work has been conformance and structure rather than features: every hardcoded user-facing string is now a translatable message behind two CI gates (section 9.4); two drag-only interactions have a keyboard route, and a set of controls that announced themselves as glyphs now have names (sections 9.2 and 9.3); and `viewer-core` is inside the section 3.3 size limits throughout (section 3.2). **One claim in the last issue was wrong** and is corrected in section 12 — the environment note that qualified every statement about test results.

Section 13 states exactly what is left — read it before quoting anything here to a customer.

Scope. The viewer as described by ADR 12 (`cde-platform`, `docs/adr/0012-viewer-as-a-standalone-product.md`).

Companions. `viewer-integration-guide.md` for how a host uses it; `viewer-extraction-inventory.md` for what still has to move; and the platform's own `docs/architecture.md`, which remains the authority on tenancy, the ISO 19650 state machine and the request lifecycle.

Contents

1. What this is, and what it is not

It is a browser-side viewer for construction documents — PDF, DXF/DWG drawings, and IFC models — with a markup, measurement, redaction and signature layer over the top, plus the server-side conversion needed to turn formats a browser cannot read into ones it can.

It is not a document management system. It does not own documents, users, projects, permissions or an audit trail. The host CDE owns all of those, and the viewer's job is to borrow a document briefly, render it, and hand back what the user drew on it.

That distinction is the whole architecture. Every design decision below follows from "the bytes belong to someone else".

2. Three layers, and the one that matters

HOST CDE	EXCHANGE	VIEWER PRODUCT
Their web interface where the viewer appears	**1. mount the viewer →** **✓ BUILT** **iframe + cde.viewer.v1**	/embed route no session, no auth guard
Their identity provider Entra / Okta / own	**2. identify the user →** **✓ BUILT** **host.init.identity — 3 untrusted fields**	viewer-core 15 files · 6 services · 6 components
Their document store SharePoint / S3 / Blob	**3. hand over the document →** **✓ BUILT** **host.init.document, or POST /api/conversions**	Conversion pipeline fetch → scan → render
Their markup store they stamp the author	**← 4. return what was drawn** **✓ BUILT** **3 markup events + 1 operation pair**	host-channel origin-checked in both directions
	⚠ NOT YET DEPLOYABLE **the allow-list exists; nothing serves the embed document**	

Figure 1 — All four exchanges now have code on both sides, checked against each other by a demo host written in plain JavaScript. The `frame-ancestors` allow-list the backend refused to have now exists — `cde.web.embed-parent-origins`, closed until a deployment names a host. What is still missing is a tier that serves the `/embed` document at all: the backend image carries no frontend, and the Kubernetes manifests route everything to the backend.

`src/viewer-core/` is the product. 3,220 lines of production TypeScript across 15 source files, with 8 spec files beside them. Nothing in it imports anything from the application — not a service, not a model, not an environment constant. When the separate repository exists, this directory is **copied, not untangled**.

It is now shaped as a library rather than a directory: `package.json`, `ng-package.json`, `tsconfig.lib.json` and a curated `index.ts`, registered in `angular.json` as a second project built by `@angular/build:ng-packagr`. The name is `@cde/viewer-core` and the licence field reads `UNLICENSED` — deliberately, so an accidental `npm publish` fails rather than quietly shipping to a public registry with no licence position. Removing that marker is a commercial decision, not a cleanup.

The application shell above it is this platform's own consumer of the viewer. It is not part of the product; it is the first integrator, and useful precisely because an awkward contract will be awkward for us first. Since the embed shipped there is a **second** consumer — `demo/`, a host application in plain JavaScript — and it is the more honest of the two, because it is the side an integrator actually writes.

2.1 The boundary is asserted, not intended

`viewer-core.boundary.spec.ts` reads every `./*.ts` in the directory through Vite's `import.meta.glob` and fails on four conditions.

Assertion	Why it exists
At least 6 source files present	A renamed directory would otherwise make every check below vacuously true
Nothing imports app/, core/, features/, shared/, environments/	The dependency that breaks extraction
Nothing reaches above the directory except ../testing/ from a spec	Catches the same thing if a folder is renamed
No production file reaches outside at all	Stops the test-helper exemption becoming a hole

Table 1 — The four boundary assertions.

One `inject(AuthService)` added in a hurry turns a copy into a migration, and **nothing else in the build would notice** — the application compiles perfectly well with the dependency pointing the wrong way. That is why this is a test and not a convention.

Components sit flat beside the services rather than in a `components/` subdirectory. The glob is `./*.ts` and the rule forbids every `../` import, so a nested component reaching back for `../viewer-state.service` would both escape the glob and trip the rule it escaped.

3. viewer-core

3.1 Services

Service	Lines	Responsibility
viewer-state.service	375	The single source of truth. Signals for document, page, zoom, rotation, active tool, selection, sidebar tab, and the version-commit token that makes panels reload after a server-side operation
markup-engine.service	372	Shape creation, geometry updates and hit testing. Pointer handling and the in-progress shape moved out to markup-drawing-session when the second surface needed them
pdf-engine.service	238	pdfjs-dist wrapper — document loading, page rendering to canvas, text-layer extraction
measurement.service	165	Scale calibration and the geometry behind length, area and radius. unitsPerPixel === 1 with unit px means uncalibrated
outline.service	147	A PDF's bookmarks and its link annotations
drawing-search.service	129	Text search across a converted drawing's labels

Table 2 — The six server-independent services.

3.2 Components

`cad-viewer` (the DXF/DWG canvas), `ifc-tree` (the model hierarchy), `outline-panel`, `page-links`, `tool-rail`, and `icon` — the last being the product's whole icon set as stroke-only 24×24 path data, so one definition sits on a light rail, a dark header and an accent-filled button without variants.

Every component in `viewer-core` is now inside the §3.3 limits. The last issue recorded `cad-viewer.component.ts` at 678 lines and called splitting it debt that wanted a runnable test suite first. The suite runs (§12), and the split happened: `cad-viewer` is 118 lines, with the viewport transform, the image lifecycle, the layer panel and the markup overlay each in their own file. `ifc-tree` went the same way, 398 to 190, with a row component and a roving tab stop beside it.

The decomposition was not cosmetic. Splitting a file forces every seam through a test, and four defects came out of these two alone: a drawing whose rotation shift had never been asserted, a layer panel whose checkbox the spec bypassed by reaching into the component, an image-viewer branch that could never have run, and model metadata computed, translated and never rendered.

> Two spec files remain over the 400-line file limit — `ifc-tree.visibility.spec.ts` (458) and `markup-drawing-session.spec.ts` (408). Recorded rather than hidden; they are the next thing to split.

3.3 What is deliberately not here

The 33 viewer components still in the application are not there by accident. Eighteen import the platform's HTTP services directly; the rest are presentational children produced by the §3.3 decomposition, and those move as they are — a page card, a signature stamp, a form-field row and a toolbar have no opinion about where their data came from.

That count rose while the coupling did not, which is the point: splitting an oversized component multiplies files, and the question "what still has to move" is answered by the eighteen, not the thirty-three. `viewer.component` is no longer among them — it was dead and has been deleted.

ADR 12's open step 4 is now closed. ADR 14 decided the rule: reads and computations are viewer-side, and anything that changes the document is a callback into the host. That covers seventeen operations, and three of them — `document.sign`, `version.create`, `version.restore` — never become viewer-side whatever else changes, because a signature over a copy the host has since replaced is worse than no signature. What remains is the work of moving these components, not the question of where they go.

4. The embed surface

New since the last issue of this document, and the reason its status line changed. The contract ADR 14 settled is now code on both sides.

`src/app/features/embed/` — 1,128 lines of production TypeScript across six files, with 811 lines of spec beside them. It is deliberately **not** in `viewer-core`: the protocol is how a host talks to a deployment of the viewer, so it belongs to the application that is deployed, while `viewer-core` stays the part that knows no backend exists.

File	What it does
embed-protocol.ts	The envelope, the message union, and the validators. No Angular, no DOM — it is the contract as types plus the predicates that enforce them
host-channel.service.ts	The postMessage boundary. Every inbound message is checked in a fixed order: origin, then event.source, then protocol envelope, then direction
embed-session.service.ts	The session state machine — ready, awaiting init, loaded, failed — and the document-open path
embed-page.component.ts	The rendering surface and the markup overlay
embed-viewer.component.ts	The /embed route component. No auth guard, deliberately — the viewer authenticates nobody
markup-wire-format.ts	Translation between the viewer's ShapeData and the protocol's Markup — the only part of the session with no session state, so it tests against the two representations directly

Table 3 — The embed implementation.

Thirteen message types run viewer → host and six run host → viewer. Four of the thirteen are lifecycle events — `viewer.opened`, `viewer.unloaded`, `viewer.markupLoaded` and `viewer.pageRendered` — added after the first integrations asked how to record what a reader did. None is required: a host can ignore every one and still open documents and collect markup. Adding them was additive within v1, which is the compatibility promise in §10 of the protocol doing the job it exists for.

> **`viewer.pageRendered` is deduplicated per document, and that is the whole point of it.** Zooming repaints a page; it is not a second reading of it. An undeduplicated stream would inflate a "pages read" figure by however many times the user changed zoom, and a host building that metric would have no way to tell. The viewer deduplicates so that every host does not have to, and so that the ones that forget are not silently wrong.

4.1 Five rules, and the order they are checked in

§5.13.8 and the protocol's own §3 give five rules that are not negotiable: never post to `'*'`, check the origin on every message rather than once, check `event.source` as well as the origin, validate in both directions, and never put a credential in a message. The first four are enforced in `host-channel.service.ts`, in that order, and a message failing any of them is dropped with the reason recorded.

> **The order is load-bearing, and so is checking `event.source`.** Origin alone is not enough: any frame on the host's page — an advertisement, a third-party widget — posts from the host's origin. Without the source check a sibling frame can drive the viewer, and nothing about the message would look wrong.

4.2 Two implementations, checked against each other

`demo/public/host-protocol.js` is an independent implementation of the host half — 237 lines of plain JavaScript, no framework, no build step, no dependency on anything in this repository. `protocol-conversation.spec.ts` then runs the **real** viewer channel against the **real** demo host through two fake windows, and asserts the conversation rather than either side's internals.

That is worth more than either side tested against a mock. A mock encodes the same assumption the implementation makes, so the two agree by construction; two implementations written from the specification disagree wherever the specification was ambiguous, which is exactly where an integrator would have disagreed with us.

> **A green test proves nothing until the thing it guards is broken on purpose.** The host-trusts-the-author rule passed its first test while broken, because the viewer never sends an author and the test asserted an outcome rather than forging the field. The test that works posts a `viewer.markupCreated` carrying `author: "Someone Else"` and asserts the host stores the name from its own session instead.

Eleven Playwright tests drive the demo in a real browser against a stub viewer on a third origin. They earn their keep: three defects no unit test could reach were found there — a captured `contentWindow` going stale across navigation, an author `display` rule beating the user agent's `[hidden] { display: none }` so a placeholder swallowed every click, and the page and the server disagreeing about the viewer's origin.

Four, now. The eleventh test exists because **the demo was missing a documented event, and the demo is what an integrator reads instead of the specification.** `viewer.selectionChanged` is in the protocol's event table and is genuinely sent by `embed-session.service.ts`, and the demo neither handled it nor summarised it in its log — so a host built by copying the reference integration would have omitted it without anything saying so. It now highlights the stored row whose markup the reader has selected, and clears the highlight when the selection goes to `null` — the half most likely to be got wrong, because a host that treats a clear as a selection leaves the last row highlighted for ever.

4.3 A documented event that nothing sent

> Checking the demo against the protocol found the drift running the other way too, and worse. **`viewer.resized` was declared in the `ViewerMessageType` union and documented in `docs/viewer-embed-protocol.md` — the file `embed-protocol.ts` names as the authority — with a payload shape of `{ contentHeightPx }` and guidance on when a host should ignore it. No code path ever sent it.**

A host implementing the published protocol would wire a handler, size its frame to the content, and wait for an event that never arrives. Nothing errors. The frame simply never resizes, and the integrator looks for the bug in their own code, because the document they are working from says the event exists.

No test could have caught it: the event nothing sends is also the event nothing asserts, so the suite was green and the protocol was wrong. What catches it is a comparison between the three places the protocol describes itself — the union, the `send` calls, and the authority's event table — which is now the `check:embed-protocol` gate in §12.2. It was verified by putting the defect back and watching it fail, and it names the consequence rather than the discrepancy: *a host that handles it waits for an event that never arrives.*

> The event is removed rather than implemented, and that is a deliberate choice with an open question behind it. The viewer paginates and scrolls internally, so "content height" has no single meaning — one page, all pages, or the current fit — and the answer is entangled with whether the viewer should ever manage its own scrolling, which changes the accessibility story around focus and scroll containers. **That decision is open; advertising the event before it is made is not a way of keeping it open.** A host that wants the frame to follow the content measures it from its own page.

4.4 What the embed cannot open

> **Only PDF renders in an embedded frame today.** `embed-session.service.ts` treats `application/pdf` as directly renderable and refuses everything else with a 415

> `conversion-required` problem naming the format. The conversion service in section 6 exists and works — the embed path simply does not call it yet. Office and IFC are therefore supported by the product and **not** supported through the embed, which is a distinction worth stating before a customer discovers it.

5. Three rendering pipelines

PDF is rendered in the browser by `pdfjs-dist` 6. Text layer extracted for search and redaction; markup drawn as SVG in a layer above the canvas so it scales with zoom without re-rasterising.

DXF/DWG cannot be rendered by the browser at all. The converter turns a DXF into SVG server-side; DWG is converted to DXF first. The result is an SVG the browser renders directly, with an invisible text layer over it for search — the same trick the PDF viewer uses, for the same reason.

IFC is rendered with `three` 0.185 as WebGL geometry, with `ifc-tree` carrying the model hierarchy alongside it. **The tree is the primary interface and the canvas is the visual layer over it**, not the other way round — a WebGL canvas cannot be made WCAG-conformant on its own, and §1A.4 requires an equivalent accessible route to the same information. Building the tree first is what makes that true rather than aspirational.

5.1 The same drawing, rendered twice, on purpose

A DXF is converted twice, by two different paths, and the reason is not obvious enough to leave undocumented.

For the viewer, geometry is drawn as SVG paths and the text is emitted as `<text>` with `fill="none"` — invisible, positioned, selectable. The browser draws the glyphs from the geometry; the invisible layer exists only so search and selection have something to hit.

For PDF export, that trick is inverted. LibreOffice discards SVG text it cannot see, so an invisible layer would produce a PDF with no extractable text at all. The print render therefore draws **no** glyph geometry (`TextPolicy.IGNORE`) and emits **visible** `<text>`, which LibreOffice turns into real, searchable PDF text.

> **The consequence that bites: calibration must measure what was actually drawn.** The viewer render includes glyph paths, so it measures the whole layout; the print render draws no text, so it measures geometry only. Using the wrong set silently shifts every label on the sheet.

A related trap, recorded because it cost a day: rendering white-on-white passes every test that checks the PDF exists, has pages, and contains text. Only rasterising the output and measuring ink coverage catches it, and that check now lives in the converter's test suite.

6. Why conversion is server-side and stays there

Office documents go through LibreOffice, DWG through the ODA File Converter, OCR through Tesseract, DXF through `ezdxf`. None of these run in a browser, and all of them are heavy native parsers on untrusted input — §5.13.10 requires them in a sandboxed, resource-limited, network-isolated worker with a timeout, out of process.

This is the constraint that rules out the otherwise-attractive design where the browser fetches straight from the customer's storage and we never see the file: **that architecture cannot render a .docx at all.**

6.1 DWG — resolved by removing the encumbered binary

Changed since the last issue. Two tools can turn binary DWG into DXF, and they had opposite licence problems.

	LibreDWG dwg2dxf	ODA File Converter
Licence	GPL-3.0	Proprietary
In the image	**No longer — removed**	No — the operator supplies it
The problem it had	We shipped it and owed every recipient corresponding source we did not provide, so any distribution of the image was a breach	We cannot ship it, so a customer install has no DWG unless they obtain one themselves

Table 4 — Two converters, opposite licence problems — and how the first one was ended.

ADR 13 was resolved on 2026-09-08 by deleting the problem rather than answering it. LibreDWG is gone from the converter image, so nothing encumbered is distributed and the licence finding closes by deletion. The image now contains **no DWG reader at all**; DWG requires an operator-supplied ODA File Converter, mounted into the container and pointed at by `ODA_PATH`.

That is a smaller product and a defensible one. It also costs nothing in fidelity — ODA is the reference implementation, and the pipeline was designed around it before LibreDWG was added as a fallback. Every other format is unaffected: DXF renders through `ezdxf` with no external tool, and Office, PDF and IFC never touched LibreDWG. The counsel questions become live again only if bundled DWG is ever wanted.

A supplied ODA is fully supported: discovered from a directory or a binary, launched inside a virtual framebuffer (it is a Qt application and opens a display even converting from the command line), probed once at startup, and reported as `odaRunnable` distinctly from `odaInstalled` — a mount missing its libraries or its execute bit is present and useless.

> `libredwgInstalled` remains in the status response, always `false`, marked deprecated with a sunset of 2027-04-01. §3.4 forbids removing a field inside an API version without at least six months' notice, and a client branching on it reads the truth — the tool is not installed.

7. Document ingress

The server-side half of exchange 3, and the one an integrator can use without embedding anything. The host mints a short-lived URL with its own credentials and posts it; the viewer fetches once, converts, and discards.

#	STAGE	WHAT HAPPENS
1	**Host mints a link**	Short-lived, single-object, your credentials. We never hold them.
2	**Submit**	POST /api/conversions { sourceUrl, targetFormat } — 202 + Location header, under one second
3	**Destination policy**	Resolves the hostname and validates the ADDRESS, not the string. Refuses loopback, link-local, 169.254.169.254, RFC 1918, .internal. Redirects are NOT followed.
4	**Bounded fetch**	Size cap enforced during the stream, not after. Connect

#	STAGE	WHAT HAPPENS
		and read timeouts.
5	**Quarantine + scan**	Magic-byte type check, then ClamAV. Not referenced by the application until clean.
6	**Convert**	Sandboxed, resource-limited, network-isolated worker, out of process.
7	**Poll**	GET /api/conversions/{jobId} — PENDING, RUNNING, then SUCCEEDED / FAILED / CANCELLED (terminal)
8	**Collect**	GET /api/conversions/{jobId}/content — streamed

Figure 2 — The conversion API, end to end. Stages 1, 7 and 8 are yours; 2 to 6 are ours. Always send an Idempotency-Key so a timed-out retry returns the same job rather than converting twice. An embedded viewer opening a PDF skips this path entirely — the browser fetches your URL directly and no server of ours sees the file.

A refused destination is a **422**, and a job belonging to another tenant is a **404** rather than a 403 — invisible, not forbidden. Every operation requires `document:convert`.

The viewer never holds a customer credential and never learns which storage platform the URL points at — SharePoint, S3, Azure Blob and GCS all reduce to the same code path. That property is what makes “integrates with any CDE” a design rather than a slogan.

The source URL is never stored. It is a bearer credential with a short life; persisting it would turn the job table into a credential store. Only its host is kept on the job record.

The address is checked twice, deliberately: cheaply on submission so an obviously wrong value fails fast with a clear message, and again against the resolved address at fetch time, which is the check that actually decides. No pattern can validate what a name will resolve to.

7.1 The gap under this, narrowed but not closed

The two halves now meet for one format. The embed opens a PDF by fetching the host’s URL directly from the browser — no server of ours is involved, which is the best possible answer for the one format that needs no conversion.

> **For every other format the halves still do not meet.** `/api/conversions` works and the embed does not call it, so an embedded viewer asked for a `.docx` or an `.ifc` returns a 415 `conversion-required` rather than converting it. The application’s own front end likewise still reads `/api/viewer/{id}` and `/api/documents/*`, which assume the platform owns the document. Wiring the embed to the conversion service is the single change that turns “PDF only” into the format list in section 6.

8. State

Angular signals throughout, with `ViewerStateService` as the single store. No NgRx, no observable soup: the viewer’s state is small, synchronous and local — zoom, page, tool, selection — and a store framework would add indirection without adding capability.

Server-side operations that rewrite the document (redaction, OCR, flatten, form fill, page rearrange) commit a **new version** and bump a reload token. Panels watch the token rather than each other, which is what lets those operations compose: each starts from the previous one’s output rather than from the untouched original.

9. Accessibility and localisation architecture

Both are procurement gates rather than preferences, and both are structural here for the same reason: a host CDE embedding this viewer inherits our conformance and our untranslated strings, and can fix neither from the outside.

9.1 The three structural decisions

- **The IFC tree is the primary data interface.** A canvas alone cannot conform. It now implements the full tree pattern — a roving tab stop, arrow navigation, `aria-level` and `aria-expanded` on each row — rather than a list of divs that looked like a tree.
- **Every drag interaction needs a single-pointer alternative** (SC 2.5.7).
- **Exports must be tagged and PDF/UA-conformant.** An inaccessible export is a product accessibility failure, and it is the most common gap found in government audits.

9.2 What the second of those cost

SC 2.5.7 was stated in the last issue of this document and not implemented. Two drag-only interactions have since been given a keyboard route.

- **Page reordering** was a CDK drag handle and nothing else. Angular's CDK drag-drop has no keyboard mode and a `<span cdkDragHandle>` cannot take focus, so a reader without a pointer could select, rotate, duplicate and delete pages but never move one — while the grip's own translator note claimed reordering worked from the keyboard. Each page now carries a pair of move buttons that name the page they move.
- **The comparison wipe** was a drag-only divider. It is a range input with the visual handle drawn over it, so it answers arrow keys, Home and End.

> **The markup layer was the remaining gap, and it is closed for creation.** Two issues of this document said no shape could be created or moved without a pointer and called it the largest accessibility item outstanding. A shape can now be drawn entirely from the keyboard, on both surfaces.

The overlay takes focus when a drawing tool is active, and announces what the keys do — the only place they are written down for a reader who cannot see the crosshair. Arrow keys move that crosshair, Shift takes the coarse step, Space presses, Enter finishes and Escape cancels. Space-then-arrows-then-Enter is the single-pointer alternative SC 2.5.7 asks for, because the arrow keys between the press and the finish are doing what the drag did. The overlay claims a key only when it used it, so arrow keys still scroll a page nobody is drawing on and Tab still leaves.

> **The sentence that named the gap was conflating two different things, and only one of them was an accessibility defect.** Creating a shape was pointer-only — that was real, and is fixed. *Moving* a committed shape is not a keyboard gap at all: there is no shape selection and no move in the viewer, for any input device. `removeShape` is the only thing that acts on a committed shape. So the second half of that claim described a feature nobody has rather than a route some readers lack, which made the accessibility gap look larger than it was and hid a plain functional gap behind it.

The keyboard drives the same state machine the pointer does. The three pointer handlers now hold nothing but coordinate extraction and hand a point to `pressAt`, `moveTo` and `release`; the keyboard moves a point and calls the same three. A separate keyboard path would have passed its own tests and still come to disagree with the pointer about which tools work, what a shape's defaults are, or where a finished one goes — which is not a hypothetical. It is the drift this session was extracted to stop, and it had already

happened once between the PDF page and the CAD drawing. One test asserts the keyboard and the pointer produce an identical shape from identical coordinates; that assertion is the design, not a detail of it.

The crosshair is a component, extracted on the second caller rather than the third. §3.3 says to duplicate twice and abstract on the third, and this is the exception that rule's own wording allows: the two callers are the two surfaces that have drifted before, over exactly this question of what works where. Drawn twice it ends up two sizes, or present on one surface and absent on the other — and a reader who cannot see it has no way to tell which of those happened.

> **What this does not do.** It is a keyboard route to *create*; it is not a screen-reader account of what is already on the page. A reader still cannot enumerate existing markup, and cannot move or resize a committed shape, because nobody can. Neither is this a substitute for the testing §1A.5 requires: these are unit tests of a state machine, and the manual keyboard pass and the screen-reader matrix remain the real gates.

The callout half of that sentence is fixed, and the sentence was wrong about why. It read: `updateShape` still has no `callout` case, so a callout box cannot be dragged at all. The first clause was true and the inference from it was not — acting on it would have changed nothing. `pointerDown` routed a callout by `isTextTool`, which answers "does this shape carry words" rather than "is this put down by a click or by a drag", so a callout took the click path and `updateShape` was never reached for one at all. Adding the missing case on its own would have been inert, which is the trap in a gap recorded with a plausible cause attached: the next person fixes the cause and ships no change. Those are two different questions and now have two predicates.

A callout has two positions — the anchor on the feature and the label clear of it — so it is now the one text tool placed by dragging: press on the detail, pull the label to where it should sit, and the words are asked for on release, which is also the only order a modal prompt permits. Before this every label sat at the seeded offset of eighty right and forty up: on a crowded drawing usually over the detail being annotated, near a page edge off the page, and no gesture moved it. A click that never moves still places a callout at that offset, so the gesture that already existed is extended rather than replaced.

> **This is not an accessibility improvement and must not be counted as one.** It adds a drag to a layer that was already wholly pointer-dependent. The label's only pointer-free outcome is the seeded offset, which is a fallback and not the equivalent SC 2.5.7 asks for.

9.3 Defects that were not on anyone's list

A pass over the panels found a class of failure worth naming, because it is invisible to an automated checker and to a sighted developer alike: **accessible-name computation takes element content over `title`.** A `<button>` whose whole content is "↺" is announced as that glyph; its tooltip is never read. Six controls announced themselves as a symbol. The fix is an `aria-label` on the button and `aria-hidden` on the glyph.

The same pass found a label pointing at a DOM id that did not exist, because the id was a PDF AcroForm field name and "Given name" has a space in it; four buttons bound to empty method bodies; required form fields marked only by an `aria-hidden` asterisk; validation messages not tied to the control they were about; and status icons announced as "white heavy check mark" beside a badge that already said the status.

A later pass found one more of the same family, and it is the one a buyer's auditor is most likely to meet: **the embedded toolbar declared `role="toolbar"` without implementing it.** That role promises arrow-key navigation over a single tab stop, and the row is a set of

ordinary buttons each with its own tab stop. A screen-reader user is told it is a toolbar, presses the arrows, and nothing moves — worse off than one told it is a group of named buttons, which is what §1A.2 means by bad ARIA being worse than none. It is `role="group"` now, which promises only what is there.

The full viewer's own command bar had already reached that conclusion and written it down in a comment two directories away. A rule recorded in a comment beside one component does not reach the next person writing another, which is why it is in this document and asserted in both components' specs.

> None of these would have been caught by axe. **This is the argument for §1A.5's position that automated checks are a floor, not conformance** — and it is worth quoting to a buyer who asks what our VPAT rests on.
>
> **One defect was on the list, and the list was wrong about it.** The conformance report in `cde-platform/docs/accessibility/` recorded SC 2.4.6 as *Does Not Support: no <h1>; heading hierarchy is incomplete.* The first clause was false when it was written — three <h1> elements already existed — and the second was true. A row that is wrong in our favour on one clause is a row nobody re-reads, so the clause that was right sat there unactioned.

What was actually wrong: **four of the seven routes had no heading of any level**, among them `/viewer/:id`, the main document viewer. A reader navigating by headings arrived at the viewer and found nothing naming the document that was open. `/login`, `/compare` and `/viewer3d/:id` were the same, and the embedded frame had an <h1> only in its *failed* state — so the state a reader actually spends time in had none.

Where a visible title already existed, it now carries the semantics rather than gaining a hidden copy beside it: the document viewer's top bar and the 3D toolbar held the page's title in a , and hold it in an <h1> now. Tailwind's preflight resets heading size and margin, so nothing moved on screen. A second, hidden heading would have been a copy that can drift from the one a sighted reader is looking at. Where no visible title existed the heading is hidden, because the alternative is worse: sign-in shows a brand lockup, which names the product rather than the task, and compare shows two file slots and a `VS` pill, none of which names the page.

> **The embedded frame is its own document, so it needs its own heading** — the host's <h1> is not inside the iframe. Hidden, because a host sized that frame to show a document and a title we injected would be chrome in their page. One detail there is load-bearing: **its visually-hidden rule is declared in the component's own styles, not borrowed from the application's utility layer.** Every other class in that component is local by design, and a visually-hidden rule that silently stops applying does not fail quietly — it puts a heading we injected on screen inside a customer's page.
>
> **Heading levels no longer skip, and the last issue of this document was wrong about why they could not be fixed.** It said the skip was "not a number to change in one template" because "the same panel component appears at more than one depth, so no single hard-coded level is correct in both places", and concluded that the route outlines needed designing — a decision rather than an edit.

That premise was false, and checking it took one grep. Each of the panels that owns an `h3` is used in exactly one place: the signature and version panels in the viewer sidebar, the summary in compare, the properties panel in the model tree. Nothing appeared at two depths. **A defect deferred on a premise nobody tested is worse than one left open with no reason given**, because the reason is what stops the next person looking.

What was actually in the way was the opposite problem — a missing level, not an ambiguous one — and it came with a second defect attached. The viewer's sidebar is a tabbed region: `sidebarTab()` renders exactly one panel at a time. Seven of its ten panels

carry no heading at all, and the region has no landmark role and no accessible name either, so a reader who chose Measure or Outline arrived at content with nothing saying which panel they were in. The two panels that *do* have a heading start at `h3`, directly under the top bar's `h1`.

One `h2` in the sidebar fixes both: it names whichever panel is showing, and it puts a level between the `h1` and those `h3`s so they become correct rather than being renumbered to paper over a gap. Its text comes from the `tabs` table the strip is already built from, so there is no new string to translate and the heading cannot disagree with the tab that selected it. Hidden, because that tab already shows the same word.

The other two routes were the same promotion made elsewhere in this section. Compare's "AI Summary" was a `<span>` whose own translator note called it the heading of the panel, and the model tree's "Model Tree" was a `<div>` whose note called it a heading while serving as the tree's `aria-labelledby` target. Both are headings now, both keep doing what they did, and the `h3`s beneath them are correct.

> `modal-dialog` keeps its `h3` deliberately. It is the dialog's accessible name through `aria-labelledby` inside `role="dialog"`, which is an isolated context announced as a dialog; renumbering it against whatever happens to be behind the dialog would be guesswork, and a level that depends on the caller is the genuine version of the problem the last issue imagined.

9.4 The labels half of SC 2.4.6

The criterion is headings *and* labels. The headings half above had been worked through twice; the labels half had never been looked at, and the conformance report said so rather than implying otherwise. This is that audit, and it is a gate now — `check:labels` — because the answer only stays true if something keeps asking.

> **Eight controls had no accessible name.** Five were labelled by a placeholder, or by nothing: both document searches, and three of the controls for naming a form field drawn on a page. A placeholder is not a label — it disappears the moment the reader types, so anyone who comes back to the field hears nothing, and voice control has no name to address it by.

Three announced as nothing at all. The sidebar toggle in the viewer's top bar, whose only content is an `aria-hidden` icon — missed by the pass that gave six other icon-only buttons a name, and relying on `title`, the weakest source in the accessible-name algorithm. The per-document status chooser in the grid, where one shared name would have announced as a grid of identical choosers. And **every link annotation in every PDF**: those are empty anchors laid over the words the document draws, so a page with fifty cross-references put fifty nameless links in the tab order. The name was already being computed for the tooltip; it simply never reached assistive technology.

Where a control repeats, each one is named so they can be told apart — the status chooser by its document, the form-field controls by position and page. A single shared name passes a naive check and still leaves a reader choosing between several identical controls.

What the audit got wrong first, and why that is in this document

The first run reported **fourteen** controls. Six were correctly labelled by means a single template cannot see, and finding that out took reading each one: a thumbnail button named by the `alt` of the image inside it; four controls in one dialog wrapped in `app-labelled-field`, which renders the `<label for>` around its content; and a file input hidden with `class="hidden"` — `display: none`, so not focusable and not in the accessibility tree at all, which is the opposite of `sr-only`, which hides visually and *does* still need a name.

> **Each of those is now an exclusion with the reason written beside it**, because an audit that cries wolf gets switched off, and then it catches nothing. The gate's own tests assert the exclusions directly, since they are the part most likely to be removed by someone tidying up.
>
> Then the reverse: fixing a traversal bug in the gate **revealed five defects it had been hiding.** It returned the first collection of children it found instead of accumulating them, so anything inside an `@empty` or a `@switch` went unread — and `@switch` keeps its cases in `groups`, not `cases`. **A false negative in a gate is worse than a false positive:** nobody is told, and the clean report is evidence of nothing.

What this gate cannot do

A label can point at `labelTarget()` while the control it names carries `[id]="controlId"` — two different expressions resolving to the same value. That is correct code, and no reader of the template alone can pair them up; five properly labelled controls in the form a reader actually fills in were reported on exactly that shape. So a template holding a bound `for` and a control holding a bound `id` is taken as associated.

> **That is a deliberate floor with a cost worth naming: a genuine mismatch between a bound `for` and a bound `id` passes this gate.** Catching it needs the rendered page. This is what §1A.5 means by automated checks being necessary and not sufficient, and it is the honest reason the manual keyboard pass and the screen-reader matrix are still the real gates rather than a formality. The gate checks that a name *exists*. Whether a name is *apt* is a judgement it does not make and should not claim to.

9.5 Localisation

Every sentence a reader sees is an `i18n` attribute or a `$localize` call carrying a stable message id and a translator note, extracted to a committed catalogue at `src/locale/messages.json` — **595 messages at this issue,** up from 589.

> **The last issue of this document said there were none left, and that was wrong by nine.** They were found by writing specs over two panels that had none, not by either gate. Three were in the version-history panel — one inside a `confirm()`, two set on an error signal — and six in the embed viewer, three of them operands of a template expression and three returned from a method. None was template *text*, which is what `check:i18n-markup` sweeps for, and none was a `$localize` call, which is what the extractor collects. A string in an expression operand or a return statement is invisible to both. The embed viewer is the worst place in the product for it: it renders inside a customer's own application, so the English appeared framed by their translated interface.

The gates have not been widened to catch this class, and saying so is more useful than implying they have. What would catch it is a lint rule against bare string literals reaching a template binding or a user-visible signal, which is a different kind of check from either existing gate. Until then the honest statement is that the catalogue is complete as far as two mechanical sweeps can see, and a string placed where neither looks will still get through.

Two gates hold it, and both run in CI. `check:i18n` regenerates the catalogue and fails on any difference, the same contract §3.5 imposes on the OpenAPI spec and for the same reason: a generated file nothing compares to its source rots, and a stale catalogue ships an untranslated string to every language at once. `check:i18n-markup` sweeps component templates for text a translator will never see.

Two things a host should know. Server-produced English still reaches the screen in a few places where only the server knows what happened — it has been removed wherever the server also sends a status code we can word ourselves, which is most of them. And the viewer ships the source catalogue, not translations: a host wanting French supplies French.

10. Security posture

Most of §5 is inherited from the platform and documented there — RLS tenant isolation, the hash-chained audit trail, PBKDF2 password storage, the §5.4 response headers. What the **viewer product** must own itself once distributed is the question this table answers.

Control	Where it lives now	Where it must live
SSRF policy on the ingress	Platform fetch/ package	Travels with the product — it is the product's own attack surface
Upload magic-byte + AV	Platform	Travels
Content Security Policy	Platform response headers; frame-ancestors is an allow-list on the embed route, 'none' everywhere else	Travels with the product. Deployment-level today; per-tenant needs a discriminator the request cannot forge
Tenant isolation	Platform RLS	Does NOT travel. The host owns tenancy; the viewer must not assume it
Audit	Platform hash chain	Host's concern; the viewer emits events, it does not store them

Table 5 — What travels with the product and what does not.

The last two rows matter most in the long run: a distributed viewer that assumes it owns tenancy would be wrong in a way that is expensive to unwind. **The third row is what matters this week.**

10.1 frame-ancestors, and what it did and did not unblock

Done. `cde.web.embed-parent-origins` names the host origins permitted to frame the `/embed` route. Every other route keeps `'none'`, and a test fails if a relaxation lands globally rather than on that route. `X-Frame-Options` is now written per route rather than globally — it has no allow-list form, so on the embed route it could only say `SAMEORIGIN` and a browser honouring it would refuse the frame however the CSP reads.

Closed by default. An empty list yields `frame-ancestors 'none'` — the same policy, byte for byte, that every other route gets. Adding the route does not on its own make anything framable, which is what made the change safe to merge before anyone had decided which customers may embed.

> **This is a security change, not a configuration change.** `frame-ancestors` is the authorisation decision about who may frame the viewer; `parentOrigin` in the handshake is only addressing — it says where to post, not who is allowed. Confusing the two would produce a viewer that any site could frame as long as it sent the right message.

Values are validated at startup, because **a `frame-ancestors` source a browser cannot parse is not a closed door** — the browser drops the unparseable source and applies what is left, so a typo silently widens the policy instead of breaking visibly. Wildcards, the `null` origin, CSP keywords offered as origins, anything carrying a path, and non-http schemes are all refused by name.

Deployment-level, not per-tenant, and that limit is deliberate. The embed route carries no credential by design, so there is no authenticated principal to derive a tenant from; deriving one from a query parameter or a header would let a caller nominate its own allow-list, which is a wildcard with extra steps. For the viewer as ADR 12 sells it — a product

a customer installs — one deployment is one customer, and that is the boundary this draws. Narrowing further needs a discriminator the request cannot forge.

Where a deployment puts the Angular build behind a separate web tier, that tier serves the document and must carry the same header — the backend's copy governs only what the backend answers. The supported way to avoid that split is 10.2.

10.2 Serving the embed document from the image

The allow-list was necessary and not sufficient: until recently nothing in the deployment served `/embed` at all, because the image carried no frontend while `k8s/ingress.yaml` routed every page request to it. ADR 15 closed that. `cde.web.app.path` points the image at a staged Angular build, and the same component then emits both the document and the policy that governs it.

Empty by default. An image with nothing staged serves no pages and answers page routes with 404 — the behaviour before ADR 15, and the right one where a web tier serves the build. The bundle is staged into the backend's build context before the image is built, because the Angular sources live in a sibling repository a Docker context cannot reach.

> **The document cannot be a static file.** §5.4 forbids `style-src 'unsafe-inline'`, and a production build carries inlined critical CSS, module scripts and a small script that promotes the deferred stylesheet — all of which need a nonce matching the header on the same response. So the document is rendered per request, with `Cache-Control: no-store`, and the build stamps a placeholder the server substitutes. Splitting the document from its header across two components is what this design exists to avoid.

Verified by running the demo host against it, end to end. The full journey works: the document opens, three pages render, markup is drawn and stored by the host, the host page is reloaded and the markup is handed back and painted, `goToPage` and `setZoom` drive the viewer, and a capability that was not granted does not render its control. No content blocked, no console errors.

> **That run found six defects that every unit test had passed over.** A policy refusing the build's own stylesheet-promoting script, so the main stylesheet never applied. `/embed` failing dependency injection before rendering anything. A pdf.js worker URL that had never resolved on any deployment. A `connect-src` with no way to name an integrator's storage. A service worker serving the entry document from cache, which defeats a per-request nonce by construction. And markup that never rendered in either direction, because the overlay bound `[innerHTML]` to SVG and Angular's sanitiser strips it — while every protocol message stayed correct, so the host's log looked perfect and the page was blank. **Run the demo against a served viewer before believing an integration works.**

11. Performance

§7.1 applies unchanged: every interactive request under a second, bulk work async with a job id returned in under a second. Conversion is bulk by definition.

Frontend budgets — LCP < 2.0 s, INP < 200 ms, CLS < 0.1, initial bundle < 250 KB gzipped — are the ones the viewer can actually breach on its own. The PDF and 3D renderers are the risk: both are lazy-loaded via dynamic `import()` so they never enter the initial bundle.

Large files never touch application memory. A 2 GB IFC streams to object storage and is processed out of process; §7.7's prohibition on `readAllBytes` over user content is absolute.

12. Build and verification

Angular 22 with TypeScript strict **and** `noUncheckedIndexedAccess`. Third-party JavaScript is bundled, never loaded from a CDN — §5.12 A08, and it is also what makes air-gapped deployment possible.

> **The Node version note has now been wrong twice, in opposite directions, and the facts are worth stating carefully because they qualify every test claim in this document.** The issue before last said `ng test` cannot run on Node 22.22.2 at all. The last issue corrected that to "npm's engine range is advisory unless `engine-strict` is set" and reported the suite running. Both are half right, and the half that matters is this: **npm's `engines` check is advisory, and the Angular CLI's own runtime check is not.** They are two different gates. `npm ci` installs happily; `ng test` then reports the detected version, states that the CLI requires v22.22.3 or v24.15.0, and exits without running anything. Whether the suite runs depends entirely on which Node is on `PATH`, not on `engine-strict`.

The development container ships 22.22.2, which is 0.0.1 below the floor, so **running the suite there requires a newer Node** — installing 24 and putting it first on `PATH` is enough, and that is how the figures below were measured. A fresh container needs it again; nothing in the repository pins it. Recorded at this length because a 0.0.1 shortfall that stops the entire test suite, with an error message about npm engines nowhere in it, is exactly the kind of thing the next person loses an hour to.

On a Node that satisfies it, `ng test` and `ng build --configuration production` both run: **2,221 specs pass, and the production build is clean.** TestBed specs are the majority of them.

That correction is load-bearing in an unwelcome direction. Several components had never been type-checked against their own templates, because `ng test` does not type-check a component no spec imports and `tsc --noEmit` does not check Angular templates at all. Only `ng build --configuration production` catches both, and running it surfaced real defects — a template referencing a signal that did not exist, another calling `new` where the Angular parser has no `new`. Both checks stay in the loop.

What remains true: the demo's end-to-end tests drive a stub viewer rather than the real one, so **the `/embed` route still has not been rendered by Angular inside a host frame on an installed deployment** (§13).

> **`test:demo` has a second trap of the same shape, and it reads as a missing install rather than a mismatch.** Playwright resolves its browser by build number, so the pinned `@playwright/test` asks for a specific one — currently 1234 — and a container that ships a different build (this one ships 1194) fails every test in the gate with *Executable doesn't exist*, followed by an invitation to run `playwright install`. Running it is the wrong move here: `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD` is set deliberately and the download would not be permitted anyway.
>
> **The remedy is already in the repository, and the last issue of this document described a workaround instead of it.** `demo/e2e/playwright.config.mjs` reads `PLAYWRIGHT_CHROMIUM_PATH` and passes it as `launchOptions.executablePath`, which bypasses the build-number lookup entirely: pointing it at the installed browser passes all ten against the one that is present. The previous note said the browser had to be made "reachable under the path the pin expects", which works and is the wrong advice — it reconstructs by hand a mechanism the config already supports, and it leaves behind container-specific directories that the next upgrade silently invalidates. The pipeline stage uses the environment variable, and falls back to installing the pinned build only when no path is given.

12.1 Coverage

§14's figures are now the gate. `angular.json` requires 90% line and 85% branch — the standard itself — where it previously required 78/78, which was a figure the suite happened to reach rather than one anybody had chosen. Measured: **90.87% line (5,727/6,302) and 85.64% branch (2,840/3,316)**, with statements at 87.9% and functions at 74.8%, both also gated at their measured values so neither can fall.

> **The denominator grew while the work was done** — from 5,375 lines to 6,175. The builder's `coverageInclude` filters rather than forces inclusion, so v8 counts a file only once something imports it, and each new spec pulls in its component's whole dependency tree. Coverage therefore *fell* before it rose: the first few specs took the reported figure from 79.4% down to 76.2% by making eight hundred previously invisible lines visible. The figure now covers substantially more of the application than the 79.4% it started from did, which is the opposite of how a rising coverage number usually reads.

The figure is a floor and not the point, and §14 says so: high coverage with weak assertions is worse than honest lower coverage. Every assertion written to reach it was checked by deliberately breaking the thing it guards and confirming it failed only the cases that name it — nineteen deliberate weakenings in all. That is also how the nine untranslated strings in section 9.5 and the `role="toolbar"` defect in section 9.3 were found. Neither was failing anything.

Where the remaining 10% sits: the PDF markup layer's pointer handling, the collaboration service's socket lifecycle, the embed page's stroke session, and `pdf-engine.service`'s rendering paths — all of them code whose behaviour is a canvas or a socket rather than a return value, and none of them untested by accident.

12.2 Gates

Each of these fails loudly rather than warning, and each exists because the thing it checks is otherwise invisible in a diff.

Gate	What it asserts
check:no-remote-code	Nothing in src/ or demo/ loads executable code from a CDN. Two roots, each with its own minimum file count, so a wrong path fails instead of passing vacuously
check:attribution	THIRD-PARTY-NOTICES.txt regenerates to exactly what is committed. It regenerates and asks git, rather than checking the file exists — the backend learned that distinction the hard way, with a shipped attribution naming a version it no longer had
check:icons	The application icons and favicon match their generator byte for byte, and the mark stays inside the maskable safe zone
check:samples	The demo's three sample documents match their generator byte for byte
test:demo	Eleven Playwright tests drive the demo host in a real browser
check:i18n	The committed message catalogue regenerates to exactly what is in the tree. It captures the extractor's stderr rather than inheriting it, because duplicate message ids are reported there and it exits 0 regardless — two source strings sharing an id means one of them ships the wrong words in every translated language
check:i18n-markup	No component template carries user-facing text a translator will

Gate	What it asserts
	never see
check:served-assets	Nothing in the built bundle is unreachable code
check:bundle-budget	The initial bundle, three.js and pdf.js each stay inside their §7.1 budget — currently 122.8 kB / 250, 187.2 kB / 200 and 149.3 kB / 160
check:embed-protocol	The embed protocol's three self-descriptions agree: every type in the ViewerMessageType union is sent by production code and has a row in the authority's event table, and nothing is sent or documented from outside it
check:labels	Every focusable control has an accessible name. A placeholder is not a label, an aria-hidden icon contributes nothing, and title is the weakest source in the algorithm
test:scripts	The gate scripts themselves have tests, so a gate cannot pass by being broken

Table 6 — The repository's own gates.

All twelve now run in CI, where four did two issues ago. The CI Node is not the container's, so the version trap above does not reach the pipeline. The platform's `Jenkinsfile` runs `npm ci`, `tsc --build --force --noEmit` and `ng test` against this repository, and every gate in the table: `check:no-remote-code`, `check:i18n`, `check:i18n-markup`, `test:scripts`, `check:icons`, `check:samples`, `check:embed-protocol` and `check:labels` in the static-analysis stage; `ng build --configuration production` then `check:served-assets` and `check:bundle-budget` in the build stage, because those two read the bundle and cannot run before it exists; `check:attribution` beside the backend's own licence gate; and `test:demo` as its own stage under Tests.

> The six that moved had existed and passed locally for some time, which is the state worth naming rather than the fix: **a gate nothing runs is documentation.** Each was written because the thing it checks is otherwise invisible in a diff, and then left where nothing could fail on it.

The count was ten rather than nine because `check:bundle-budget` was missing from the table above — it has existed and passed since the budgets were set, and a gate absent from the list of gates is one nobody will think to wire up.

Two of the placements are load-bearing rather than tidy. `check:served-assets` and `check:bundle-budget` sit after the production build in the same shell, so neither can be reached without the artifact it reads — a check that passes itself over a missing bundle reports success for every build that never produced one. And `check:attribution` is both halves of the frontend's licence assurance, not just a file comparison: it refuses a forbidden licence (§2.1) or one on neither list, and refuses to write `THIRD-PARTY-NOTICES.txt` over a violation, so it cannot report a clean file for a tree that is not clean. The pipeline's own list of gates it does not provide is one line shorter as a result.

> One correction to the `tsc` line in the last issue: the pipeline passes `--build --force`, not a bare `--noEmit`. `tsconfig.json` carries `"files": []` and project references, so `tsc --noEmit` type-checks nothing at all and exits 0 — proven by putting a type error in a production file and watching it pass. **A gate that cannot fail is worse than no gate, because it is quoted.**

`check:attribution` passes, and the last issue of this document was wrong to say otherwise — twice over. It reported the gate failing on two packages whose licences sit on neither §2.1's allowed nor its forbidden list, `caniuse-lite` (CC-BY-4.0) and `lru-cache` (BlueOak-1.0.0), and concluded that this blocked §17.6's release checklist until somebody ruled on it. Somebody had already ruled on it: the determination is recorded in `docs/licences.md` §3.6, dated 2026-09-24, and both licences are allowed — BlueOak-1.0.0 is a plain-language equivalent of MIT and BSD-2-Clause that additionally carries a patent grant, which §17.3 prefers to silence on patents, and CC-BY-4.0's only obligation is attribution, which `THIRD-PARTY-NOTICES.txt` discharges. The gate now writes 119 components across seven licences and reports the file current.

Its account of *why* was also wrong, and in a way worth correcting rather than deleting. It said both packages arrive "only through `@angular/build` and `@angular/cli`, which are devDependencies". They do not: they reach the production closure through `@angular/localize`, which is correctly a production dependency because `$localize`'s runtime is imported at runtime even though the Babel tooling underneath it runs only at build time. That distinction is the whole reason the gate counted them, and "the lockfile marks them production-reachable" described the symptom as though it were an accident of the lockfile rather than a true fact about the dependency. **§17.6's release checklist is not blocked by this, and no longer blocked by the gates either** — the six that nothing called are now pipeline stages. What remains outstanding against that checklist is in section 13, and none of it is a frontend licence question.

13. What this architecture does not yet have

Stated plainly, and shorter than it was. The gap between "the viewer works" and "a CDE can integrate it" used to be a set of undecided contracts; it is now a set of unfinished jobs, in rough order of what blocks what.

No document has opened in a host frame on an installed deployment. Section 10.2. The pieces are each verified — the protocol against an independent host, the allow-list against its own tests, the served document against a real browser — and the whole has never run. This is the next thing to do and the thing most likely to surface something nobody predicted.

The library and the application are built from one repository. The image serves the Angular build staged into the backend's context, which works and means the frontend is versioned with the backend that carries it. A customer wanting the viewer without the platform still has no artefact of their own; that waits on `viewer-core` being published, below.

The embed opens PDF and nothing else. Section 4.3. The conversion service exists; the embed path does not call it.

Collaboration has no identity in an embed. Live cursors and presence ride a STOMP socket authenticated by the page session, and an embedded viewer has no session. ADR 14 accepted this as the loose end that "no identity" leaves; it is unsolved rather than decided.

THE PLATFORM'S OWN API — unchanged	THE EMBED PATH — built
Priya, at Asite host identity	Priya, at Asite known only to Asite
↓ **blocked**	↓
Annotation.author @ManyToOne User	host.init.identity display name · subjectId · capabilities
↓	↓

THE PLATFORM'S OWN API — unchanged	THE EMBED PATH — built
OUR users table she must exist as a row	Nothing persisted the viewer stamps no author
Integrating through this API means shadow-provisioning every host user into our database.	The host stays the system of record, and stamps the author itself when it stores the markup.

Figure 3 — Both paths exist in the code. `/api/annotations` still keys an author to a row in our own user table, and always will for this platform's own users. The embed path never reaches it: the viewer emits a markup with no author at all, and the host attributes it from the session it already holds.

No external document identity on our own `Document`. The protocol carries `externalId` end to end, so an embedding host never needs one — but the platform's own `Document` entity still has no such column, so anything integrating through `/api/documents` keeps a map between its id and our numeric one.

35 endpoints across 15 files — 11 services and 4 components — derived by walking imports transitively from every viewer component, viewer service and `viewer-core` file. Two groups drive the design: **five are content**, and are exactly what the ingress replaces, and **seventeen are document operations**, which ADR 14 assigned to the host. The remainder is markup, identity and collaboration.

> That total has been wrong twice, in the same direction. It was "seven", then "26 across 9", and both were undercounts — the first from grepping directories instead of following imports, the second from a resolver that turned `role.service` into `role.ts` by replacing the extension rather than appending one, so every `*.service.ts` import silently failed to resolve and its endpoints were never counted. **A count that resolves imports must fail loudly when one does not resolve**, or it reports the surface it could see as the surface that exists.
>
> **And it is not settled yet.** `viewer-extraction-inventory.md` §2 gives a total of 35 across 11 services and 4 components, but its own table lists ten services, and its group sizes — content 5, document operations 17, markup 7, identity 2 — sum to 31. Two of those three figures must be wrong. The groups quoted above are the ones the inventory states unambiguously and that a design decision actually turns on; **the partition needs re-deriving before it goes in front of a customer**, and no number here should be quoted as a complete breakdown until it does.

A library in shape, never built as one — and now with a known reason. `viewer-core` has its manifest, its ng-packagr configuration and its own `angular.json` project, so `ng build viewer-core` is a real target. The last two issues gave two different wrong accounts of it: first that it could not run for the Node reason, then that the Node reason was wrong so nobody had run it.

It has now been run, on a Node that satisfies section 12's floor. **It fails, with 53 errors, all of them the same one:** `Cannot find name '$localize'`, across ten of the `viewer-core` files. The cause is one line — `tsconfig.app.json` carries `"types": ["@angular/localize"]`, which is what puts the `$localize` global in scope, and `tsconfig.lib.json` carries `"types": []`, which clears it. The library sources are the application sources, so every `$localize` call in them is an unresolved name under the library's own compiler options.

> **That line has deliberately not been changed**, because adding the type would make the build succeed and produce an artefact nobody has decided to publish — and it would decide something on the way. A library published with `$localize` calls still in it requires every consumer to run Angular's localize transform over our code, which is a packaging contract to offer a host CDE, not a compiler setting to flip while correcting a document. The alternatives — extracting the strings to an injected catalogue, or shipping the library

> pre-localised per locale — are the same decision seen from two other sides. The target exists, it does not build, the reason is understood and small, and what it waits on is the publish decision the `UNLICENSED` marker already records as untaken.

Accessibility evidence. The artefacts exist in `cde-platform/docs/accessibility/` — an accessibility statement, a VPAT 2.5 INT conformance report, and a screen-reader matrix — and all three say the same thing about themselves: the statement is a draft not fit to publish, every criterion in the ACR reads Not Evaluated or Does Not Support, and no screen-reader pass has been run. **What is missing is not the document; it is the evidence the document is supposed to cite.** There is still no axe run, no Lighthouse budget, and no gate in the pipeline to produce either.

> **The axe gate is blocked rather than merely undone, and it is worth saying on what.** §1A.5 names axe-core via Playwright, and `@axe-core/playwright` is not in this repository's dependency tree. Adding it needs a version, and §0.1 says to resolve one through the Context7 MCP and — if Context7 is unavailable — to say so and ask rather than guess. Context7 has been unavailable throughout this work. Pinning a version from memory is exactly the thing that rule forbids, and an accessibility gate introduced by guessing at a dependency would be a §0.1 violation shipped in the name of §1A. **This needs the dependency approving under §0.3 with a resolved version; it is not an engineering problem.**
>
> **Separately: a committed markup shape cannot be moved or resized by anyone.** There is no shape selection in the viewer and no move — `removeShape` is the only operation on a committed shape, so a misplaced annotation has to be deleted and drawn again. This was previously recorded as half of an accessibility gap, which hid it: it is not a keyboard gap, because no input device can do it. Keyboard creation does not address it and was not meant to.

What has changed is the ground underneath them. The defects in sections 9.2 and 9.3 were real conformance failures and they are fixed, with a keyboard-operable test pinning each one — so an ACR filled in today would read differently from one filled in at the last issue. It would still read mostly Not Evaluated, because a test asserting an `aria-label` exists is not a screen-reader pass and must never be reported as one (§1A.5). **The ACR has not been updated, and should not be until someone runs the matrix.**

The embed makes this worse rather than inheriting it. Focus crossing a frame boundary, printing from inside a frame, and 200% zoom in a host-sized frame are all new surfaces that need testing afresh, and some of them will be worse than the standalone viewer. §1A is explicit that without a current conformance report the public-sector buyers this product targets cannot be bid to at all — and an independent audit has the longest lead time of anything on this list.

Brand and copyright. The application mark is our own work and deliberately generic; the product has no name or logo that has been through trademark clearance, and `LICENSE` and `NOTICE` still carry a placeholder where the copyright holder's legal entity goes. An obviously unset placeholder is the right state until the entity is decided — a plausible-looking wrong name is a false statement of ownership that survives into every distribution.

> The shape of this list has changed since the last issue. The embed and identity contracts were **decisions nobody had taken**; they are taken, specified and implemented. What remains is work nobody has done — and the first item is one response header.